



Crear Entidades `Usuario` y `Rol` para Seguridad con JPA

Introducción a la Guía

En esta sección vamos a preparar nuestra aplicación para que Spring Security **ya no utilice usuarios en memoria**, sino que pueda **leer usuarios y roles desde una base de datos** usando JPA (Java Persistence API).

Para lograrlo, vamos a crear las **clases de entidad** `Usuario` y `Rol`, además de configurar un **DAO** y un **servicio personalizado**.

Esto permitirá que, al iniciar sesión, la aplicación valide los datos directamente en las tablas de MySQL de manera profesional y segura.

¡Comencemos! 

◆ Paso 1: Crear la clase `Rol.java`

- Mapeamos la tabla `rol` de la base de datos.
 - Usamos anotaciones `@Entity`, `@Table(name="rol")`, `@Id`, `@GeneratedValue`.
 - Implementamos `Serializable`.
 - Agregamos atributos:
 - `idRol`
 - `nombre`
 - Usamos `@NotEmpty` para validar que `nombre` no esté vacío.
 - Agregamos los constructores vacío y con todos los argumentos, los métodos `getters` y `setters` y `toString()`.
 - El código de las clases se agrega al final de esta guía.
-

◆ Paso 2: Crear la clase `Usuario.java`

- Mapeamos la tabla `usuario`.
 - Relación **Uno a Muchos** (`@OneToMany`) con `Rol`.
 - `idUsuario` es la llave primaria (`@Id`, `@GeneratedValue`).
 - Atributos:
 - `username`
 - `password`
 - `roles` (relación a `List<Rol>`)
 - También usamos validaciones `@NotEmpty`.
-

◆ Paso 3: Crear el DAO `UsuarioDao.java`

- Creamos la interfaz `UsuarioDao` que **extiende** `JpaRepository<Usuario, Long>`.
- Añadimos un método personalizado:

```
Usuario findByUsername(String username);
```

- Esto permitirá buscar usuarios automáticamente filtrando por su nombre.
-

◆ Paso 4: Crear el Servicio `UsuarioService.java`

- Creamos un servicio `@Service("userDetailsService")`.
- Implementamos la interfaz `UserDetailsService`.
- Inyectamos `UsuarioDao` con `@Autowired`.

- Implementamos el método `loadUserByUsername(String username)`:
 - Buscamos el usuario.
 - Convertimos sus **roles** a objetos de tipo `GrantedAuthority`.
 - Devolvemos un objeto `User` de Spring Security con username, password y roles.

? ¿Por qué implementamos la interfaz `UserDetailsService`?

Cuando trabajamos con **Spring Security** y queremos **autenticar usuarios desde una base de datos**, Spring necesita una **forma estándar** de obtener la información de los usuarios (nombre de usuario, contraseña y roles).

- ♦ La interfaz `UserDetailsService` define justamente ese contrato.

Esta interfaz tiene un único método:

```
UserDetails loadUserByUsername(String username) throws UsernameNotFoundException;
```

- ♦ ¿Qué significa esto?

Que **Spring Security llama automáticamente** este método cuando alguien intenta iniciar sesión, **pasándole el username** que el usuario ingresó en el formulario de login.

- ♦ Nuestra tarea al implementar `UserDetailsService` es:

- Buscar en la base de datos si existe un usuario con ese nombre.
- Si existe, **convertirlo** a un objeto que Spring Security pueda entender (un objeto `UserDetails`).
- Si no existe, lanzar una excepción para indicar que el usuario no fue encontrado.

✅ Todo quedó configurado para que **Spring Security pueda autenticar usuarios desde la base de datos** usando JPA y las nuevas entidades.

✨ Código completo

🔧 Clase `Ro1.java`

```
package gm.domain;

import java.io.Serializable;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotEmpty;
```

```
@Entity
@Table(name="rol")
public class Rol implements Serializable{
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idRol;

    @NotEmpty
    private String nombre;

    public Rol() {
    }

    public Rol(Long idRol, String nombre) {
        this.idRol = idRol;
        this.nombre = nombre;
    }

    public Long getIdRol() {
        return idRol;
    }

    public void setIdRol(Long idRol) {
        this.idRol = idRol;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public String toString() {
        return "Rol{" + "idRol=" + idRol + ", nombre=" + nombre + '}';
    }
}
```

 Clase Usuario.java

```
package gm.domain;

import java.io.Serializable;
import java.util.List;
import jakarta.persistence.*;
import jakarta.validation.constraints.NotEmpty;

@Entity
@Table(name="usuario")
public class Usuario implements Serializable{
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idUsuario;

    @NotEmpty
    private String username;

    @NotEmpty
    private String password;

    @OneToMany
    @JoinColumn(name="id_usuario")
    private List<Rol> roles;

    public Usuario() {
    }

    public Usuario(Long idUsuario, String username, String password, List<Rol> roles) {
        this.idUsuario = idUsuario;
        this.username = username;
        this.password = password;
        this.roles = roles;
    }

    public Long getIdUsuario() {
        return idUsuario;
    }

    public void setIdUsuario(Long idUsuario) {
        this.idUsuario = idUsuario;
    }
}
```

```
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public List<Rol> getRoles() {
    return roles;
}

public void setRoles(List<Rol> roles) {
    this.roles = roles;
}

@Override
public String toString() {
    return "Usuario{" + "idUsuario=" + idUsuario + ", username=" + username + ", password=" + password + ", roles=" + roles + '}';
}
}
```



Clase UsuarioDao.java

```
package gm.dao;

import gm.domain.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioDao extends JpaRepository<Usuario, Long>{
    Usuario findByUsername(String username);
}
```



Clase UsuarioService.java

```
package gm.servicio;

import java.util.ArrayList;
import gm.dao.UsuarioDao;
import gm.domain.Rol;
import gm.domain.Usuario;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service("userService")
public class UsuarioService implements UserDetailsService{

    @Autowired
    private UsuarioDao usuarioDao;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Usuario usuario = usuarioDao.findByUsername(username);

        if(usuario == null){
            throw new UsernameNotFoundException(username);
        }

        var roles = new ArrayList<GrantedAuthority>();

        for(Rol rol: usuario.getRoles()){
            roles.add(new SimpleGrantedAuthority(rol.getNombre()));
        }

        return new User(usuario.getUsername(), usuario.getPassword(), roles);
    }
}
```

✨ ¿Qué sigue?

En el próximo paso vamos a:

- Modificar `SecurityConfig.java`.
- Configurar para que **ya no lea usuarios en memoria**, sino desde la **base de datos** usando `UsuarioService`.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)